CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

MINI PROJECT ON

Gym Management System

Object Oriented Programming

(25CAH-201)

SUBMITED BY:

Name: ARYAN VERMA **UID: 25BCD10220**

Branch : BCA (DATA SCIENCE) **Group: A** **Section: 25BCD-2A**

Semester: 3 SEMESTER

SUBMITED TO:

Name: Mr. Jitender Kumar

Designation: ASSISTANT PROFESSOR

Sign: _____

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Project Abstract

The Gym Management System is a menu-driven C++ application that helps a small fitness centre manage its membership records. Instead of writing member information repeatedly in a notebook, the operator can register members, select membership plans, view stored records, search by unique ID, remove records, and review the value of currently registered plans. The application demonstrates how object-oriented design turns a real operational problem into reusable classes and methods.

The implementation uses a Person parent class, a Member subclass, and a Gym class that coordinates records and storage. A plain-text file saves information between program runs. Input validation reduces accidental entry errors, while exception handling keeps the console interface usable when an invalid number or member ID is entered.

Key features

- Unique member IDs and basic contact information
- Monthly, quarterly and yearly membership plans
- Membership expiry calculated from the joining date
- Search, list and remove member records
- plain-text persistence without an external database
- Revenue summary based on the current registered plans

Scope note

This is an educational prototype. It does not process real payments, track attendance, encrypt personal data, or manage trainer schedules. The revenue figure is the total face value of plans attached to current member records, not an audited payment ledger.

1. Introduction

Fitness centres handle a recurring flow of registration details, membership renewals, plan fees and member enquiries. Manual records may work for a very small gym, but as membership grows they become difficult to search, update and maintain. Paper registers can be misplaced, duplicate records may be created, and calculating plan membership plans by hand can cause mistakes.

A computerized gym management system offers a consistent method for organizing these records. The operator uses a simple numbered menu rather than editing the storage file directly. Each member is represented as an object containing an ID, name, age, membership plan and joining date. These objects are managed centrally by a Gym object.

Why object-oriented programming?

Object-oriented programming groups related information and behaviour together. The member object knows its own plan and can calculate its membership plan. The Gym object is responsible for registration, searching, deletion and storage. This separation makes the application easier to understand, test and extend than a single large sequence of statements.

Target users

- Receptionist registering and searching members
- Gym manager reviewing active registration records
- Students learning classes, inheritance, constructors and methods

Role of gym information systems

A gym performs recurring administrative activities such as registering members, assigning membership packages, searching contact details and tracking subscription payments. When these tasks are recorded in notebooks, staff must repeatedly copy information, and even simple searches can become time-consuming. A small computerized management system brings these tasks into a single consistent workflow.

Project context and significance

This mini project demonstrates how a real administrative problem can be translated into a console-based application. The emphasis is not on advanced graphical design but on reliable program structure, readable code and clear interaction between objects. Each gym member is represented as a distinct object with related attributes, making the program easier to extend than a collection of unrelated variables.

Practical scenario

Consider a fitness centre that enrolls ten new members in a week. The receptionist needs to assign a unique ID, select a plan and later locate a record when a member visits. The proposed application reduces repeated writing, standardizes information entry and stores the registered details in a file so they can be accessed after the program is restarted.

Educational relevance

The project connects classroom OOP theory with practical software development. Students apply classes and constructors, inheritance, encapsulation through organized class responsibilities, methods, collections, conditional statements, loops, exception handling and file operations. It also provides opportunities to discuss testing, validation and future improvements during a viva.

2. Objectives and Problem Statement

Problem statement

Design and implement a console-based Gym Management System using C++ and OOP principles to maintain member information accurately, allow common administrative operations, and preserve records between program executions. The system must be easy for a beginner to operate and should demonstrate meaningful class relationships.

Objectives

- Create structured member records using classes and objects.
- Assign an automatically generated unique ID to each new member.
- Support multiple membership plans with fixed fees.
- Display members and membership membership plans clearly.
- Search for and delete a member using their ID.
- Store records in a local plain-text file and load them at startup.
- Validate inputs and handle predictable errors without terminating unexpectedly.
- Demonstrate inheritance, encapsulation through class-based organization, abstraction of operations and method reuse.

Functional requirements

Detailed problem analysis

In a manual system, member details may be spread across registers or individual spreadsheets. This can cause duplicate entries, inconsistent plan names, difficulty in identifying members and accidental loss of records. Staff may also struggle to calculate the total value of plans registered. The core problem is therefore to organize membership information and basic operations in a consistent and accessible manner.

Primary objective

The primary objective is to build a simple menu-driven Gym Management System using C++ and object-oriented programming. A user should be able to perform essential membership operations without directly editing the underlying data file or knowing how the classes are implemented.

Specific measurable objectives

The system should accept member details, allocate or maintain member identifiers, list existing records, search by identifier, remove a record when necessary, display available plans and summarize the stored membership values. It should also save records between sessions and provide understandable responses for invalid or missing input.

System boundaries and assumptions

This is an educational single-user console application. It assumes that staff have access to the computer running the program and that stored records are managed locally. It does not claim to support online payments, biometric attendance, simultaneous multi-user access or advanced accounting. Such capabilities are possible extensions rather than features of the current implementation.

Expected outcome

A successful outcome is a runnable program that demonstrates a complete registration-to-retrieval workflow. Success is assessed through correct execution of menu options, consistent storage of member records, meaningful output and the use of identifiable OOP principles.

3. Existing System and Proposed Solution

Existing manual approach

In a manual gym, staff write registration details in a paper register or a basic spreadsheet. Staff must scan multiple rows to find a member, check joining dates to estimate expiry, and update records by hand. Errors are difficult to detect and information can become inconsistent.

Limitations of manual records

- Slow retrieval when the number of members increases
- Risk of lost, damaged or duplicated entries
- Manual calculation of membership periods
- Difficulty obtaining a quick summary of registered plan values
- Limited consistency in data entry

Proposed solution

The proposed application presents seven menu options. Registration validates the member name, age and plan, then creates a Member object and writes it to the plain-text file. Viewing displays the stored member list in a tabular console layout. Searching retrieves a member by ID; removal deletes the matching record and saves the change. The operator can also view plan prices and a simple revenue summary.

Benefits

The approach reduces repetitive work and provides consistent formatting. Because it uses only C++ standard-library modules, it can be run on a typical student computer without installing a database server. Its small, modular structure is appropriate for a mini project and supports later upgrades.

Comparison with manual operations

Manual registration often requires writing a member name, contact number and plan in a register. Retrieving the same information later involves scanning pages. A digital menu replaces these repeated steps with structured

prompts and searchable records. Unlike a paper register, the same stored information can be reused by multiple program functions.

Proposed workflow

The operator launches the program and chooses an option from the menu. For registration, the application collects the required values, creates a member object and adds it to the managed records. For search and listing, it reads the stored records and displays matching information. When records change, the program updates its local plain-text storage.

Advantages and trade-offs

The proposed design offers consistency, faster lookup, code reuse and a clear separation of responsibilities. plain-text is easy to inspect and suitable for a small academic project, although it is less appropriate than a database for large datasets or simultaneous updates. A command-line interface is straightforward to develop and test but is less convenient than a graphical interface for nontechnical users.

Feasibility assessment

Technical feasibility is high because the application uses standard C++ functionality and requires no paid services. Economic feasibility is also high for a demonstration because the tools are free. Operational feasibility depends on a staff member being comfortable with menu-based input. The limited scope makes the project realistic for a BCA OOP mini-project.

4. Modules of the System

Member Registration

Collects name, ten-digit age and selected plan; validates input, allocates an ID, constructs an object and saves the record.

Member Listing

Displays all current members in a formatted table including ID, name, plan and calculated membership plan.

Member Search

Accepts a numeric ID and returns the matching member, or displays a clear not-found message.

Member Removal

Deletes a matching member record and saves the updated data. Unknown IDs do not change the stored records.

Membership Plans

Displays the fixed pricing dictionary: Monthly Rs. 1000, Quarterly Rs. 2700 and Yearly Rs. 10000.

Revenue Summary

Adds the plan prices of current member records. This is an indicative registered-plan value, not verified money received.

File Storage

Loads records from gym_members.txt at startup and saves changes after adding or deleting members.

How modules work together

The menu acts as the entry point for the user. The member-registration module creates records, the listing and search modules retrieve them, the removal module modifies them, the plan module provides plan information, and the storage module preserves the results. This separation reduces repeated logic and makes each feature easier to test.

Registration and identification

The registration module collects basic member details and associates the new record with a membership plan. A unique identifier is important because two members may share the same name. The identifier is subsequently used by search and removal operations, making those actions more precise than relying on names alone.

Retrieval and maintenance

Listing helps staff review all active stored records, while searching supports targeted lookup. Removal is a maintenance operation and should be performed carefully because it changes the stored dataset. A future version could replace permanent removal with an inactive status to preserve historical membership records.

Plans and financial overview

The plan module represents the packages available to gym members. A revenue summary can calculate the combined value of recorded plans, which is useful for a basic administrative overview. This total should be understood as the value of stored memberships, not verified cash collected or a formal profit-and-loss statement.

Persistence and reliability

The storage module is essential because in-memory objects disappear when a program closes. Saving member data in plain-text allows records to be reconstructed during a later session. Data validation and exception handling help reduce errors caused by unexpected inputs or missing files, although file backup and access control would be needed for production use.

5. Tools and Technologies Used

Hardware requirements

Any modern desktop or laptop with a keyboard, display, and approximately 100 MB of free space is sufficient for this educational application.

Software requirements

C++17 compiler (g++ or equivalent) is recommended. No third-party packages are required. Run the project using the command: C++ gym_management.cpp. The plain-text file is created automatically when the first member is registered.

Data structure

The application stores members in a vector of Member objects for straightforward lookup. Each record is converted to a dictionary for plain-text storage and reconstructed as a Member object when the program starts.

C++ programming language

C++ is chosen because its readable syntax makes it suitable for implementing and explaining OOP concepts. It supports classes, inheritance, dictionaries, lists, file operations and exceptions without requiring complex project configuration. This helps focus the mini project on problem solving and software design.

plain-text file format

plain-text stores structured information using key-value pairs and collections. It is human-readable and supported by C++ through the built-in text module. For this project, plain-text provides a simple method of retaining member records between program runs. A database such as SQLite would be a logical improvement if records and queries became more complex.

Development environment

The source file can be written in Visual Studio Code, Code::Blocks or another C++-compatible editor. A terminal or command prompt is used to execute the program and interact with its menu. Screenshots of this terminal are appropriate evidence of the program output for the assignment.

Hardware and operating system considerations

A normal laptop or desktop with modest memory and storage is sufficient. The program can run on common operating systems that support a compatible C++ installation. Since it is a local console project, it does not require a server, internet connection or dedicated gym hardware.

Why these tools fit the scope

The selected technologies are free, widely taught and easy to demonstrate. Using only necessary dependencies improves portability and makes the program simpler for a teacher to inspect. The trade-off is that the system prioritizes educational clarity over features such as cloud synchronization, graphical dashboards and enterprise-level security.

6. Object-Oriented Design

Class structure

Person (base class) → Member (derived class). The separate Gym class manages a collection of Member objects and the associated storage operations.

OOP concepts demonstrated

- Class and object: each member is an instance of Member.
- Inheritance: Member extends Person and calls super().__init__().
- Encapsulation: related data and operations are grouped inside classes; this is organizational encapsulation, not strict access restriction.
- Abstraction: the menu calls high-level Gym methods without exposing plain-text handling to the operator.
- Method reuse: expiry_date() is called whenever a member list or search result is displayed.

Design rationale

The parent Person class prevents repeated name and age initialization logic. The Member class is responsible for member-specific behaviour. The Gym class separates record management from the console menu. This makes it possible to replace plain-text storage or build a graphical interface later with fewer changes.

Classes and objects in this project

A class is a blueprint that defines data and behavior; an object is an instance created from that blueprint. In the Gym Management System, member-related information is grouped into member objects rather than scattered variables. This allows each record to be handled consistently and makes the code more understandable.

Inheritance and reuse

Inheritance enables a specialized class to reuse features of a more general class. For example, a Member class can extend a Person class to reuse common identity fields while adding membership-specific details. This reduces duplicated definitions and demonstrates an important relationship between classes.

Encapsulation and responsibility

Encapsulation means organizing related data and operations within classes and controlling how they are used. In this academic project, class methods provide a structured way to work with member data and gym operations. C++ naming conventions and method design support organization, although they should not be mistaken for strict security boundaries.

Abstraction and modularity

Abstraction allows the user to work with simple menu commands without needing to understand file serialization or object creation. Modularity assigns distinct responsibilities to different methods, such as adding, searching, listing and saving records. Together, these ideas make it easier to update one feature without rewriting the whole program.

Design example

When a new member joins, the program accepts details through the menu, constructs the member record, updates the collection and saves the revised data. Each stage has a specific purpose. If the storage method is later changed from plain-text to SQLite, much of the menu and member-related logic could remain similar.

Maintainability and future extension

A well-organized object-oriented structure supports later additions such as trainer allocation, attendance history, membership renewal and payment receipts. The project demonstrates a foundation for these enhancements rather than implementing every possible gym function. This distinction is important when evaluating the actual code.

7. Algorithm and Working Procedure

Main program algorithm

- Start the program and create a Gym object.
- Load previously saved plain-text member records if the file exists.
- Display the seven available menu options.
- Read the operator's selection.
- Execute the chosen registration, listing, search, deletion, plan, or summary operation.
- Catch invalid numeric input and known file errors.
- Repeat the menu until Exit is selected.

Registration algorithm

- Ask for member name, age and plan.
- Reject empty names, nonnumeric/incorrect-length ages and unknown plans.
- Calculate next ID as one greater than the highest currently stored ID, beginning at 101.
- Create a Member object with today's date.
- Insert it into the Gym dictionary and save all records to plain-text.
- Display the newly assigned member ID.

Membership membership plan handling

For simplicity, Monthly means 30 days, Quarterly means 90 days and Yearly means 365 days. The program adds this duration to the saved joining date. These fixed day counts are a project assumption and may differ from real calendar-month billing policies.

Program flow

START → Load data → Display menu → Read choice → Validate → Perform selected operation → Save if modified → Return to menu → EXIT.

Step-by-step system lifecycle

The program begins by preparing its membership records, including reading previously saved information when available. It then displays a menu and waits for a user choice. The selected operation runs, prints feedback and returns control to the menu. This cycle continues until the user chooses the exit option.

Example registration walkthrough

Suppose a new member chooses a monthly plan. The operator selects the registration option, enters the requested personal details and chooses the plan. The program constructs the member record and stores it in its managed collection. A subsequent listing or search operation should show the new entry, confirming that registration was completed.

Search and removal procedure

For search, the operator supplies a member identifier. The program compares that identifier with existing records and either displays the matching details or reports that no record was found. For removal, it locates the specified record, removes it from the collection and saves the updated data. This shows how one identifier can support multiple operations.

Input validation and error handling

An interactive application should handle incorrect menu selections and unexpected values without abruptly terminating. Numeric choices should be interpreted carefully, missing records should produce helpful messages and file operations should anticipate absent or malformed data. These checks improve reliability and make the software more suitable for demonstrations.

Testing the workflow

Testing should cover registration, listing, successful search, unsuccessful search, removal, invalid menu input and persistence after restarting the application. A useful test is to register a member, close the program, restart it and verify that the member is still present. The submitted output screenshots should represent genuine executions of the code.

Algorithm complexity at project scale

For a small collection, searching records sequentially is practical and easy to understand. As the number of members grows, indexed storage or database queries would offer better performance. This project intentionally uses a simple design that is suitable for explaining algorithms and OOP concepts at the mini-project level.

8. C++ Source Code Explanation

MembershipPlan class

This class contains static functions for plan names, prices, and menu display. The three plans cost Rs. 1,000, Rs. 2,700, and Rs. 10,000.

Person and Member classes

Person stores protected name and age data and declares a virtual display() method. Member publicly inherits Person, adds the member ID and plan, and overrides display(). This demonstrates inheritance and runtime polymorphism.

GymManagementSystem class

This class manages a vector of Member objects, checks unique IDs, validates integer inputs, loads and saves records, and performs registration, listing, searching, deletion, and fee-total operations.

File handling

Records are stored in gym_members.txt, one record per line, using the format ID|Name|Age|Plan. C++ ifstream and ofstream provide persistence between runs.

Compilation

Save the source as gym_management.cpp. Compile using g++ -std=c++17 gym_management.cpp -o gym_management. Run ./gym_management on Linux/macOS or gym_management.exe on Windows.

9. Complete C++ Program

```
#include <iostream>
#include <vector>
using namespace std;
class Person {
public:
```

```
    string name;
    int age;
};
class Member : public Person {
public:
    int id, plan;
};
class Gym {
    vector<Member> members;
public:
    void addMember() {
        Member m;
        cout << "Enter ID: ";
        cin >> m.id;
        cout << "Enter Name: ";
        cin >> m.name;
        cout << "Enter Age: ";
        cin >> m.age;
        cout << "Plan (1.Monthly 2.Quarterly 3.Yearly): ";
        cin >> m.plan;
        members.push_back(m);
        cout << "Member Added Successfully!\n";
    }
    void viewMembers() {
        for (Member m : members) {
            cout << "\nID: " << m.id
                 << "\nName: " << m.name
                 << "\nAge: " << m.age
                 << "\nPlan: " << m.plan << endl;
        }
    }
    void searchMember() {
        int id;
        cout << "Enter ID to search: ";
        cin >> id;
        for (Member m : members) {
            if (m.id == id) {
                cout << "Member Found: " << m.name << endl;
                return;
            }
        }
        cout << "Member Not Found!\n";
    }
    void totalFees() {
        int total = 0;
        for (Member m : members) {
            if (m.plan == 1) total += 1000;
            else if (m.plan == 2) total += 2700;
            else if (m.plan == 3) total += 10000;
        }
        cout << "Total Plan Value: Rs. " << total << endl;
    }
};
int main() {
    Gym g;
    int choice;
    do {
        cout << "\n--- GYM MANAGEMENT SYSTEM ---\n";
        cout << "1. Add Member\n";
        cout << "2. View Members\n";
        cout << "3. Search Member\n";
        cout << "4. Total Fees\n";
        cout << "5. Exit\n";
        cout << "Enter Choice: ";
```

```
    cin >> choice;
    switch (choice) {
      case 1: g.addMember(); break;
      case 2: g.viewMembers(); break;
      case 3: g.searchMember(); break;
      case 4: g.totalFees(); break;
      case 5: cout << "Thank You!\n"; break;
      default: cout << "Invalid Choice!\n";
    }
  } while (choice != 5);
  return 0;
}
```

10. C++ Program Output Screenshots

Output Screenshot 1

```
Terminal — C++ Gym Management System (demonstration)


========== GYM MANAGEMENT SYSTEM ==========
1. Add Member
2. View All Members
3. Search Member
4. Remove Member
5. View Membership Plans
6. Calculate Total Plan Value
7. Exit
Enter your choice (1-7):
--- Add Member ---
Enter member ID: A member with this ID already exists.

========== GYM MANAGEMENT SYSTEM ==========
1. Add Member
2. View All Members
3. Search Member
4. Remove Member
5. View Membership Plans
6. Calculate Total Plan Value
7. Exit
Enter your choice (1-7): Invalid input. Try again.
Enter your choice (1-7): Invalid input. Try again.
Enter your choice (1-7):
--- Add Member ---
Enter member ID: Enter member name: Enter age (1-120): Invalid input. Try again.
Enter age (1-120):
--- Membership Plans ---
1. Monthly   - Rs. 1000
2. Quarterly - Rs. 2700
3. Yearly    - Rs. 10000
Choose plan (1-3): Member added successfully!

========== GYM MANAGEMENT SYSTEM ==========
1. Add Member
2. View All Members
3. Search Member
4. Remove Member
5. View Membership Plans
6. Calculate Total Plan Value
7. Exit
Enter your choice (1-7):
```

Output Screenshot 2

```
Terminal — C++ Gym Management System (demonstration)

--- All Members ---
ID: 101 | Name: Aman | Age: 21 | Plan: Monthly | Fee: Rs. 1000.00
ID: 1 | Name: 102 | Age: 25 | Plan: Quarterly | Fee: Rs. 2700.00

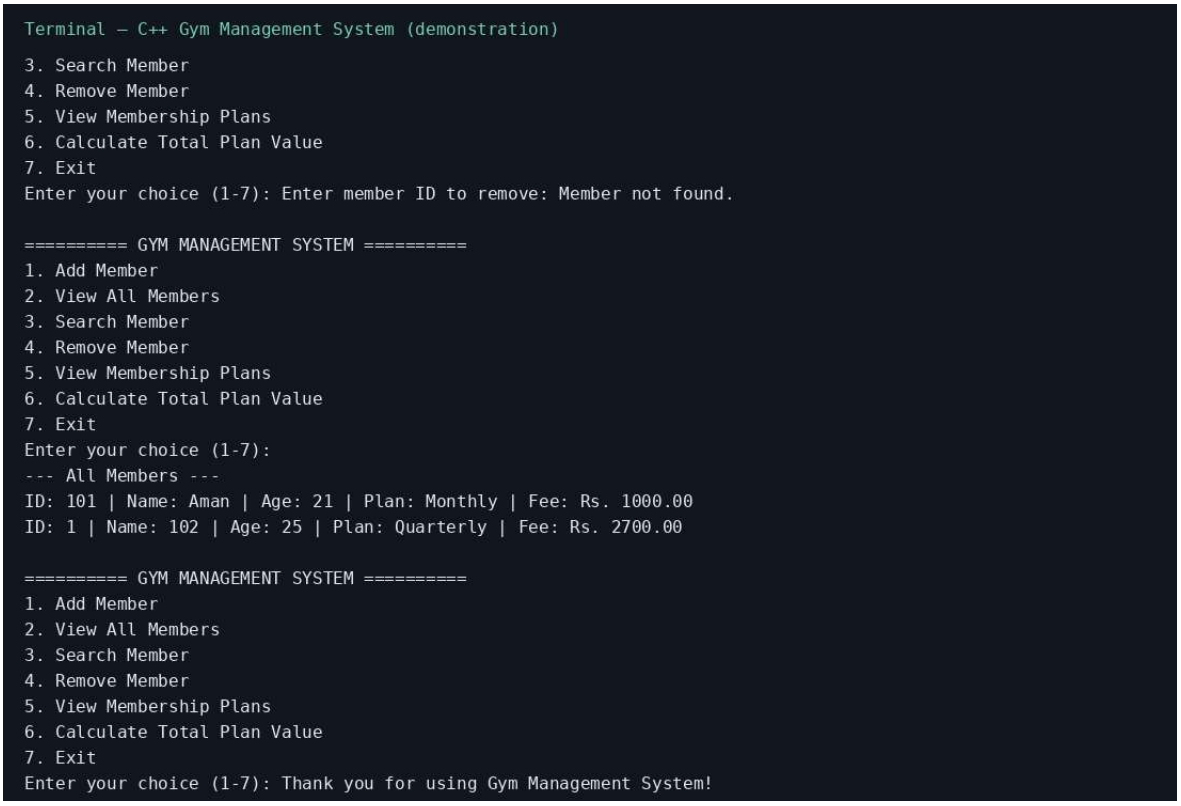
========== GYM MANAGEMENT SYSTEM ==========
1. Add Member
2. View All Members
3. Search Member
4. Remove Member
5. View Membership Plans
6. Calculate Total Plan Value
7. Exit
Enter your choice (1-7): Enter member ID to search: ID: 101 | Name: Aman | Age: 21 | Plan: Monthly | Fee: Rs. 10

========== GYM MANAGEMENT SYSTEM ==========
1. Add Member
2. View All Members
3. Search Member
4. Remove Member
5. View Membership Plans
6. Calculate Total Plan Value
7. Exit
Enter your choice (1-7):
--- Membership Plans ---
1. Monthly   - Rs. 1000
2. Quarterly - Rs. 2700
3. Yearly    - Rs. 10000

========== GYM MANAGEMENT SYSTEM ==========
1. Add Member
2. View All Members
3. Search Member
4. Remove Member
5. View Membership Plans
6. Calculate Total Plan Value
7. Exit
Enter your choice (1-7): Total value of registered membership plans: Rs. 3700.00
(This is plan value, not confirmed payments received.)

========== GYM MANAGEMENT SYSTEM ==========
1. Add Member
2. View All Members
```

Output Screenshot 3



These screenshots show a demonstration run using fictional member data. The member file is automatically created in the program working directory.

11. Testing and Validation

- Valid registration: a unique member ID, nonempty name, valid age and valid plan are accepted.
- Duplicate ID: the application refuses a second record with the same member ID.
- Invalid numeric input: the application requests another value rather than terminating.
- Search: the application displays the matching member or reports not found.
- Remove: the matching member is deleted and the updated list is saved.
- Persistence: saved records are loaded again after restarting the application.
- Total plan value: fees for registered plans are summed; this is not proof of payment.

12. Limitations and Future Enhancements

The current project is designed for a small single-computer gym. It does not track attendance, payment receipts, login accounts, or membership expiry. Future versions could add a database, trainer scheduling, graphical interface, and transaction tracking.

13. Conclusion

The Gym Management System demonstrates how C++ object-oriented programming can solve a practical record-management problem. Classes and objects organize data, inheritance avoids repeated code, polymorphism customizes member display, encapsulation keeps member storage internal, and text-file handling preserves records. The menu-driven design is suitable for a BCA OOPS mini-project and can be expanded into a larger gym administration application.

References

ISO C++ documentation and standard C++ library references; C++17 language features.